ACCENTURE

Resources + Focus + Code Bank

What to study **MORE**, official links for each resume topic, and **copy-ready code answers**

Priority key: **CRITICAL** = asked 90% times, master code + speech. **HIGH** = asked 60%, know 1 example. **MEDIUM** = know 1-line defense. Accenture OA weight: coding + pseudocode > theory. Interview weight: your 3 projects + Python + SQL + API.

1. Focus Matrix — Where to Spend Your Few Days

Topic	Priority	Focus MORE on (exam reality)
CampusPilot OCR+LLM pitch	CRITICAL	60-sec pipeline recital + 1 bug story + ReportLab. They will spend 5 min here.
FastAPI + REST + status codes	CRITICAL	Layered arch, 409 vs 404 vs 422, ?tech= filter, TestClient. Live code likely.
Python (GIL, decorator, set/dict)	CRITICAL	Jaccard with sets, Decimal, try/except, list vs dict. OA coding is Python.
SQL + Postgres joins	CRITICAL	INNER/LEFT, GROUP BY/HAVING, 409 check query. They ask to write 1 query live.
pocketF Decimal + burn-rate	HIGH	Why float fails, 9 error types list, SAFE/ALERT formula + edge cases.
UrTeam Jaccard + auth	HIGH	Formula + why not ML, 403 owner check, dual DB. Shows design sense.
pytest + Docker + Git	HIGH	1 pytest example, image vs container, compose up. Accenture loves delivery discipline.
React + fetch + hooks	HIGH	useEffect fetch, props/state, Recharts. Know 1 snippet.
Pandas/Numpy/Matplotlib	MEDIUM	groupby, vectorization, chart gen. 1-liners enough.
OpenCV/Tesseract params	MEDIUM	grayscale→blur→threshold, PSM 6, 300 DPI. Don't over-deep.
HF LLMs temp/tokens	MEDIUM	temp 0 + JSON validation, no fine-tune. Honest scope.
IF + SHAP (SIH)	MEDIUM	1-line IF idea + SHAP explains. Mark as proposed.
Aptitude + Pseudocode	CRITICAL	Speed drills daily. OA cutoff decides interview call.

2. Python — [CRITICAL]

- Sets/dicts for Jaccard + frequency; decorators; exception handling per-row; GIL implication

Learn from: Python Official Tutorial | Real Python | Decimal Docs

```
# two pointers + freq + decorator
def reverse(s):
a=list(s); i,j=0,len(a)-1
while i<j: a[i],a[j]=a[j],a[i]; i+=1; j-=1
return ''.join(a)
from collections import Counter
def first_unique(s):
c=Counter(s)
for i,ch in enumerate(s):
if c[ch]==1: return i
return -1
import functools,time
def timer(fn):
@functools.wraps(fn)
def w(*a,**k):
t=time.time(); r=fn(*a,**k); print(fn.__name__, round(time.time()-t,3)); return r
return w
```

Q: Reverse string without [::-1]?
A: Use two pointers to show logic, then mention slice shortcut. Complexity O(n).

Q: What is __init__ vs __str__?
A: __init__ constructs, __str__ pretty-prints. I used __str__ for debugging SQLModel rows.

3. SQL + PostgreSQL + SQLModel + Alembic — [CRITICAL]

- Write joins + GROUP BY/HAVING live; explain 409 check; ACID; migration commands

Learn from: Postgres Docs | SQLModel Docs | Alembic Tutorial

```
-- duplicate check (your 409) + monthly spend
SELECT email, COUNT(*) FROM student GROUP BY email HAVING COUNT(*)>1;
SELECT date_trunc('month',date) AS m, SUM(amount)
FROM txn GROUP BY m ORDER BY m;
-- alembic
-- alembic revision --autogenerate -m "add team_id"
-- alembic upgrade head
```

Q: LEFT JOIN vs INNER?
A: INNER only matches; LEFT keeps all left rows with NULLs. I list students even without team via LEFT JOIN.

4. FastAPI + REST — [CRITICAL]

- Draw routes→services→models; status codes; ?tech= filtering; TestClient

Learn from: FastAPI Docs | REST Tutorial | HTTP Statuses MDN

```
from fastapi import FastAPI, HTTPException
from fastapi.testclient import TestClient
app=FastAPI()
@app.get('/ideas')
def list_ideas(tech: str|None=None):
return {'tech':tech}
client=TestClient(app)
def test_dup():
r=client.post('/students',json={'email':'a@x.com'})
assert r.status_code in (200,201,409)
```

Q: Why 409 not 400 for duplicate?
A: 400=bad syntax, 409=conflict with state. Client knows to change email, not fix JSON.

5. React + Vite + Tailwind + Recharts — [HIGH]

- useEffect fetch pattern; props vs state; Tailwind utilities; Recharts LineChart

Learn from: React Docs | Vite Guide | Tailwind Docs | Recharts Docs

```
import {useEffect,useState} from 'react';
export default function Ideas(){
const [ideas,setIdeas]=useState([]);
useEffect(()=>{
fetch(`${import.meta.env.VITE_API}/ideas?tech=python`)
.then(r=>{if(!r.ok) throw new Error(r.status); return r.json();})
.then(setIdeas).catch(console.error);
},[]);
return <ul>{ideas.map(i=><li key={i.id}>{i.title}</li>)}</ul>;
}
// Recharts
// <LineChart data={burn}><Line dataKey="balance" /><XAxis dataKey="date"/></LineChart>
```

Q: How does React talk to FastAPI?
A: fetch(VITE_API+'/ideas') → JSON → useState → render. Handle !ok → error state. CORS enabled on backend.

6. Pandas + NumPy + Matplotlib (pocketF analytics) — [MEDIUM-HIGH]

- read_csv + groupby burn-rate; vectorized ops; savefig charts

Learn from: Pandas Docs | NumPy Docs | Matplotlib Guide

```
import pandas as pd
import matplotlib.pyplot as plt
from decimal import Decimal
df=pd.read_csv('txns.csv',parse_dates=['date'])
daily=df.groupby('date')['amount'].sum()
burn=float(daily.tail(7).mean())
days_left=float(Decimal('5000'))/burn if burn>0 else float('inf')
plt.plot(daily.index,daily.values); plt.savefig('burn.png',bbox_inches='tight')
```

Q: Pandas vs SQL?
A: SQL for storage/filter; Pandas for in-memory analytics (resample, rolling avg) before charting.

7. OpenCV + Tesseract + ReportLab (CampusPilot) — [MEDIUM]

- Preprocess chain; PSM; confidence filter; ReportLab table

Learn from: OpenCV Docs | Tesseract Docs | ReportLab Docs

```
import cv2, pytesseract
img=cv2.imread('seat.png')
gray=cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
blur=cv2.GaussianBlur(gray,(5,5),0)
th=cv2.adaptiveThreshold(blur,255,cv2.ADAPTIVE_THRESH_GAUSSIAN_C,cv2.THRESH_BINARY,31,2)
txt=pytesseract.image_to_data(th,config='--psm
6',output_type=pytesseract.Output.DATAFRAME)
conf=txt[txt.conf>60] # keep high-confidence boxes
```

Q: PSM 6 vs 3?
A: 6=uniform block (seating tables), 3=auto. Wrong PSM = garbled rows.

8. Hugging Face LLMs + Scikit-learn + SHAP — [MEDIUM]

- Prompt→JSON + validation; temp 0; IF anomaly score idea

Learn from: HF Transformers | Scikit-learn | SHAP Docs

```
from transformers import pipeline
llm=pipeline('text-generation',model='microsoft/Phi-3-mini-4k-instruct')
prompt="Extract JSON {roll,room,time} from:\n"+ocr_text+"\nReturn ONLY JSON."
out=llm(prompt,max_new_tokens=256,temperature=0.0)[0]['generated_text']
# then json.loads + regex validate roll pattern
from sklearn.ensemble import IsolationForest
clf=IsolationForest(contamination=0.02).fit(X_train) # -1 = anomaly
scores=clf.decision_function(X_test)
```

Q: Fine-tuned?
A: No — zero-shot prompt + schema validation. Fine-tune needs labeled data + GPU; overkill here.

9. Docker + pytest + Git — [HIGH]

- Dockerfile + compose up; fixture + parametrize; branch workflow

Learn from: Docker Get Started | pytest Docs | Git Docs

```
# Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn","main:app","--host","0.0.0.0","--port","8000"]
# compose
# services: {api: {build: ., ports:["8000:8000"]}, db: {image: "postgres:16",
volumes:["pg:/var/lib/postgresql/data"]}}
import pytest
@pytest.mark.parametrize("a,b,exp",[({"p"},{"p"},1.0), (set(),set(),0.0)])
def test_jaccard(a,b,exp):
from math import isclose
j=len(a&b)/len(a|b) if (a or b) else 0.0
assert isclose(j,exp)
```

Q: Container vs image?
A: Image blueprint; container running instance; volume persists DB.

10. OA Practice Links (Daily 1 hr)

IndiaBix — Aptitude + logical + verbal topic-wise MCQs with timer

GFG Aptitude — Short tricks + company-wise sets

LeetCode Easy — Do: Two Sum, Valid Parens, Missing Number, Merge Intervals

HackerRank Python — Practice input()/split() fast parsing like OA

GFG Pseudocode — Dry-run loops/bitwise/recursion

11. What to Focus MORE — Final Verdict

If only **10 hours** left: **4h** OA coding + pseudocode drills (cutoff), **3h** speak CampusPilot + FastAPI + SQL aloud with code above, **2h** pocketF Decimal/burn + UrTeam Jaccard/auth, **1h** HR intro + why Accenture + 1 STAR story. Interviewers forgive shallow OpenCV/LLM theory if your **pipeline + API + DB + tests** story is crisp and honest about scope (prototype, no fine-tune). Demo beats jargon.

Keep both PDFs + GitHub open in interview. Offer: 'I can show the 409 handler / Jaccard function / burn chart live.' That one line converts interview to selection.

12. Per-Resource Focus Map — Exact Subtopics (Study / Skip)

How to read: STUDY = do hands-on + speak aloud. SKIP = ignore for this interview. TIME = suggested mins. Interview line = what to say.

Resource [time]	STUDY vs SKIP	Interview line
Python Tutorial docs.python.org/3/tutorial/ [45 min]	STUDY: Sec5 list/dict/set/comprehension, Sec7 input/split, Sec8 try/except/else, Sec9 class/__init__/inheritance. SKIP: functional programming, venv deep, asyncio.	Say: sets for Jaccard, dict Counter for freq, try/except per CSV row.
Real Python + Decimal realpython.com + docs.python.org/3/library/decimal [20 min]	STUDY: Decimal('0.1') exact, quantize 2 places, f-string format. SKIP: fractions, statistics module.	Say: float 0.1+0.2!=0.3 so money uses Decimal.
FastAPI Docs fastapi.tiangolo.com [60 min]	STUDY: path/query params, Pydantic EmailStr/min_length, HTTPException 409/404/422, /docs, TestClient. SKIP: OAuth2, websockets, background tasks.	Say: routes thin, services pure, 409 duplicate, ?tech= filter.
REST + MDN Status restapitutorial + MDN HTTP Status [20 min]	STUDY: GET/POST/PUT/PATCH/DELETE + 200/201/400/404/409/422/500. SKIP: caching, HATEOAS.	Say: 409 conflict not 400; 422 validation.
Postgres + SQLModel + Alembic postgresql.org + sqlmodel + alembic [60 min]	STUDY: INNER/LEFT JOIN, GROUP BY+HAVING, UNIQUE email, ACID, revision --autogenerate + upgrade head. SKIP: replication, triggers, partitioning.	Say: SQLite dev, Postgres prod, migrate never drop.
React.dev + Vite react.dev + vitejs.dev/guide [45 min]	STUDY: useState/useEffect fetch, props down/events up, key={id}, VITE_API env. SKIP: redux, context, memo, SSR.	Say: fetch FastAPI JSON to state then render + error state.
Tailwind + Recharts tailwindcss + recharts.org [20 min]	STUDY: flex/gap/rounded/text-sm utilities; LineChart+Line+XAxis dataKey. SKIP: custom config, animations.	Say: inline utilities for 8 components; burn chart via Recharts.
Pandas + NumPy + Matplotlib pandas/numpy/matplotlib docs [40 min]	STUDY: read_csv parse_dates, groupby sum, tail mean burn, vectorized ops, savefig. SKIP: multi-index, merging complex.	Say: SQL stores, Pandas analyzes, Matplotlib reports.

OpenCV + Tesseract + ReportLab docs.opencv + tesseract-ocr + reportlab [30 min]	STUDY: cvtColor grayscale, GaussianBlur, adaptiveThreshold, --psm 6, conf>60, Platypus Table. SKIP: SIFT, training Tesseract, custom fonts.	Say: preprocess + PSM tuning fixed skewed scans.
HF + Scikit + SHAP huggingface + scikit-learn + shap [30 min]	STUDY: temp 0 JSON prompt + validation, train_test_split, IsolationForest(-1), SHAP one-liner. SKIP: fine-tuning, GPU training.	Say: zero-shot no fine-tune; IF isolates anomalies, SHAP explains.
Docker + pytest + Git docs.docker + pytest + git-scm [45 min]	STUDY: FROM/WORKDIR/COPY/RUN/CMD, compose api+db+volume, fixture+parametrize, branch/add/commit/push. SKIP: swarm, k8s, hooks.	Say: compose up runs all; 63 tests prove layers.
OA: IndiaBix + GFG + LeetCode + HRank indiabix + gfg + leetcode + hackerrank [daily 60 min]	STUDY: %, P&L, speed, seating/Venn + Two Sum/Parens/Missing/Anagram + stdin split parse. SKIP: Hard DP/graphs.	Say: speed 1 min/Q, easy-first coding.

Remaining resources covered same way: NumPy vectorization (no loops), Matplotlib savefig bbox_inches, Vite env import.meta.env, Alembic downgrade, Git pull --rebase — each needs 1 example only, not full docs. Total extra time ~2 hrs. Priority order top-to-bottom in table.

13. Updated Final Verdict — All Topics

Do rows 1-5 fully (4 hrs), rows 6-8 with 1 code each (2 hrs), rows 9-12 skim + 1 line (1 hr). You now have **every resource + exact subtopic + code** in these 2 PDFs. Revise table above night before.